Capstone Project — Delhivery Last-Mile Delay Intelligence Pipeline

Program: Artificial Intelligence and Machine Learning Certification Program · E&ICT Academy, IIT Roorkee **Duration:** 14 days · **Total marks:** 100 · **One deliverable:** a single public GitHub repository (one branch, `main`)

Scenario

You have joined the regional operations analytics team of **Delhivery**, one of India's largest logistics networks. Every parcel is booked with a promised delivery time (Express 36 hours, Standard 60 hours, Economy 96 hours). The regional operations head has three open problems and no pipeline that answers any of them:

1. **How long will a parcel really take?** Planners need a delivery-hours estimate for each parcel at dispatch time.
2. **Which parcels will miss their promise?** Ops can place a proactive customer call or escalate to a hub manager, but only for parcels flagged *before* they are late. A missed delayed parcel costs more than an unnecessary call.
3. **Which hubs need which playbook?** Each of the 24 hubs is labelled Tier 1, 2 or 3, but nobody has checked whether the label matches how the hub really behaves.

You will answer all three in **one connected repository**. The cleaning and feature code you write in Part A feeds the regression in Part B and the classifier in Part C; the classifier from Part C and the hub segments from Part D are combined by a final scoring script that turns a batch of new parcels into an action list. No later Part may use a number it did not compute itself or receive from an earlier Part.

> ***The data is synthetic.*** *Delhivery's real shipment records are never used. Every row in this brief's dataset is produced by the generator script below; the brand is only the setting for the decisions.*

Submission Guidelines (read first)

- **Submit exactly one public GitHub repository link.** The repository's `README.md` is the front door and carries all written answers. There are no per-Part submissions.
- **All work must live on one branch — `main` — and the link you submit must point to `main`.** The accepted link forms are the repository root URL `https://github.com/<user>/<repo>` (only when `main` is the repository's default branch) or `https://github.com/<user>/<repo>/tree/main`. Check on the repository page that the default branch is `main` before you submit. A link to any other branch, or a link that does not clearly resolve to `main`, **will not be graded**.
- **Textual deliverables only.** Code is submitted through the repository link and every result is a CSV file or Markdown text. **No images, screenshots, PDFs, slides, presentations, videos or audio are required or accepted** (do not commit any such files; none is graded). You may draw charts while exploring, but nothing you draw is a deliverable.
- **No paid or account-gated service is needed.** Everything uses free, open-source Python libraries (pandas, NumPy, scikit-learn, joblib) installed with `pip`, and runs fully offline. Reference numbers in this brief were verified on pandas 2.2.3 and 3.x with scikit-learn 1.5.2 and 1.9.x; any recent version inside that range is fine. `requirements.txt` must pin the exact versions you used (see Part A, task 1), because a pickled model can only be loaded safely by the scikit-learn version that wrote it.
- **Library documentation may be consulted** for exact argument or attribute names. The design decisions, the numbers and the explanations must be your own.
- **Money is in Indian Rupees (INR).** Write amounts as `INR 1,121.50` or `1,121.50 Rupees` in the README; never use a dollar sign for money.
- **Reproducibility is graded.** Every script must run top to bottom on a clean clone, in the order given in the README, using only files in the repository. Wherever this brief names a `random_state`, use that exact value.
- **Pass bands.** Numbers marked *exact* must match exactly. Numbers with a *band* are results from the pinned configuration in this brief (the stated split seed and the stated column order); most bands were also stress-tested across other random splits and other column orders, but the bands that depend on the split itself (for example the F-score margin of the feature-signal check) are guaranteed only for the pinned split. Use the pinned `random_state` values exactly as written; with them, a correct implementation stays inside the band. If one of your numbers is outside a band, check your code first.
- **Original work.** You may discuss approaches with peers, but the code and the README must be written by you.

- **Deadline.** 14 calendar days from the day this brief is issued. A suggested pace: Part A in days 1–3, Part B in days 4–6, Part C in days 7–10, Part D and the README in days 11–14. Part D plus the README is the heaviest writing load, so draft each README section as soon as its Part is finished instead of leaving all the writing for the last four days.

Required repository layout

```
<repo>/
├── README.md
├── requirements.txt
├── generate_data.py                # the provided script, committed verbatim
├── data/
│   ├── hubs.csv  raw_shipments.csv  new_shipments.csv     # generator output, committed
│   └── modelling_table.csv                                # written by part_a_clean_eda.py
├── src/
│   └── shipment_prep.py            # shared constants + clean_shipments + build_features
├── part_a_clean_eda.py
├── part_b_regression.py
├── part_c_classification.py
├── part_d_segments.py
├── package_model.py
├── predict_delay.py
├── models/                         # delay_model.pkl, delay_threshold.csv, feature_order.csv, impute_values.csv
└── outputs/                        # every CSV named in this brief
```

Provided data — `generate_data.py`

Commit the script below **verbatim** as `generate_data.py` and run `python generate_data.py` once from the repository root. It must print `24 1224 48` and write three files into `data/`. Do not edit any constant and do not hand-edit any generated file; all cleaning happens in code in Part A.

```
"""Synthetic shipment data for the Delay Intelligence capstone.
Run once:  python generate_data.py   (writes data/hubs.csv, data/raw_shipments.csv, data/new_shipments.csv)
Do not edit any constant: every number in the brief was measured on this exact output."""
import os
import numpy as np
import pandas as pd

HUBS = [
    ("HUB01", "Mumbai", "West", 1, 1500), ("HUB02", "Delhi", "North", 1, 1600),
    ("HUB03", "Bengaluru", "South", 1, 1450), ("HUB04", "Hyderabad", "South", 1, 1350),
    ("HUB05", "Chennai", "South", 1, 1300), ("HUB06", "Pune", "West", 1, 1250),
    ("HUB07", "Kolkata", "East", 1, 1200), ("HUB08", "Ahmedabad", "West", 1, 1150),
    ("HUB09", "Jaipur", "North", 2, 950), ("HUB10", "Lucknow", "North", 2, 900),
    ("HUB11", "Chandigarh", "North", 2, 850), ("HUB12", "Indore", "West", 2, 820),
    ("HUB13", "Nagpur", "West", 2, 780), ("HUB14", "Coimbatore", "South", 2, 740),
    ("HUB15", "Kochi", "South", 2, 700), ("HUB16", "Surat", "West", 2, 650),
    ("HUB17", "Patna", "East", 3, 520), ("HUB18", "Ranchi", "East", 3, 480),
    ("HUB19", "Guwahati", "East", 3, 450), ("HUB20", "Raipur", "East", 3, 420),
    ("HUB21", "Bhubaneswar", "East", 3, 390), ("HUB22", "Dehradun", "North", 3, 350),
    ("HUB23", "Siliguri", "East", 3, 310), ("HUB24", "Agartala", "East", 3, 270),
]
DIST_MU = [180, 320, 480]      # mean lane distance (km) for hub tier 1 / 2 / 3
DIST_SD = [50, 90, 150]
STOPS_MU = [55, 70, 85]        # mean stops on the rider route for hub tier 1 / 2 / 3
HUB_EFFECT = [0, 2, 4]         # extra hours for hub tier 1 / 2 / 3
# How a hub really operates (1 = metro-like, 2 = regional, 3 = stretched). Four hubs operate unlike their tier label.
OPERATING_TIER = {"HUB07": 2, "HUB13": 3, "HUB15": 3, "HUB22": 2}
BASE_HOURS = {"Express": 10, "Standard": 28, "Economy": 58}


def simulate(n_per_hub, seed):
    """Clean shipments, n_per_hub rows for each of the 24 hubs, in hub order."""
    np.random.seed(seed)
    hubs = pd.DataFrame(HUBS, columns=["hub_id", "hub_city", "region", "hub_tier", "hub_daily_capacity"])
    n = n_per_hub * len(hubs)
    op_tier = [OPERATING_TIER.get(h, t) for h, t in zip(hubs["hub_id"], hubs["hub_tier"])]
    tier = np.repeat(op_tier, n_per_hub) - 1
    hub_id = np.repeat(hubs["hub_id"].values, n_per_hub)
    distance = np.clip(np.random.normal(np.array(DIST_MU)[tier], np.array(DIST_SD)[tier]), 15, 1500).round(1)
    stops = np.clip(np.random.normal(np.array(STOPS_MU)[tier], 10), 20, 140).round().astype(int)
    weight = np.clip(np.random.lognormal(0.6, 0.7, n), 0.1, 40).round(2)
    value = np.random.lognormal(7.0, 0.8, n).round().astype(int)
    service = np.random.choice(["Express", "Standard", "Economy"], n, p=[0.2, 0.55, 0.25])
    pay = np.random.choice(["Prepaid", "COD"], n, p=[0.6, 0.4])
    rain = (np.random.rand(n) < 0.18).astype(int)
    day = np.random.randint(0, 90, n)
    hour = np.random.randint(6, 22, n)
    minute = np.random.randint(0, 60, n)
    ts = pd.Timestamp("2026-07-01") + pd.to_timedelta(day, unit="D") + pd.to_timedelta(hour * 60 + minute, unit="m")
    sale = ((ts >= pd.Timestamp("2026-09-15")) & (ts < pd.Timestamp("2026-09-29"))).astype(int)
    weekend = (ts.dayofweek >= 5).astype(int)
    hours = (np.array([BASE_HOURS[s] for s in service]) + 0.03 * distance + 0.10 * stops
             + np.array(HUB_EFFECT)[tier] + 0.12 * weight + 4 * rain + 7 * sale
             + 3 * (pay == "COD") + 2 * weekend + 3 * (hour >= 18)
             + 12 * sale * (service == "Express") + 8 * (distance > 400) * rain
             + np.random.normal(0, 4, n)).round(2)
    return pd.DataFrame({
        "hub_id": hub_id, "service_type": service, "payment_mode": pay,
```

```
        "parcel_weight_kg": weight, "distance_km": distance, "declared_value": value,
        "n_stops": stops, "rain_flag": rain, "sale_event_flag": sale, "dispatch_time": ts,
        "label_print_batch": np.random.randint(1000, 9999, n),
        "scanner_build_no": np.random.randint(1, 40, n),
        "tape_roll_no": np.random.randint(1, 500, n),
        "sticker_serial_mod": np.random.uniform(0, 1, n).round(3),
        "shelf_slot_no": np.random.randint(1, 300, n),
        "delivery_hours": hours,
    })


def make_messy(df, seed, n_missing, n_case, n_sentinel, n_dupes, id_start):
    """Turn a clean table into a raw export: text columns, gaps, duplicates, a sentinel."""
    np.random.seed(seed)
    df = df.sample(frac=1.0, random_state=seed).reset_index(drop=True)
    n = len(df)
    df.insert(0, "shipment_id", ["SHP" + str(id_start + i) for i in range(n)])
    df["dispatch_time"] = df["dispatch_time"].dt.strftime("%Y-%m-%d %H:%M")
    df["declared_value"] = df["declared_value"].map(lambda v: "\u20b9" + format(v, ","))
    for col, k in [("distance_km", n_missing[0]), ("parcel_weight_kg", n_missing[1]),
                   ("declared_value", n_missing[2]), ("payment_mode", n_missing[3])]:
        df.loc[np.random.choice(n, k, replace=False), col] = np.nan
    pick = np.random.choice(n, n_case, replace=False)
    shapes = np.random.randint(0, 3, n_case)
    for i, s in zip(pick, shapes):
        v = df.at[i, "service_type"]
        df.at[i, "service_type"] = [v.upper(), v.lower(), "  " + v + " "][s]
    if n_sentinel:
        df.loc[np.random.choice(n, n_sentinel, replace=False), "parcel_weight_kg"] = 999.0
    if n_dupes:
        df = pd.concat([df, df.iloc[np.random.choice(n, n_dupes, replace=False)]], ignore_index=True)
    return df


if __name__ == "__main__":
    os.makedirs("data", exist_ok=True)
    hubs = pd.DataFrame(HUBS, columns=["hub_id", "hub_city", "region", "hub_tier", "hub_daily_capacity"])
    hubs.to_csv("data/hubs.csv", index=False)
    raw = make_messy(simulate(50, 2604), seed=7, n_missing=(36, 24, 12, 18),
                     n_case=120, n_sentinel=5, n_dupes=24, id_start=10001)
    raw.to_csv("data/raw_shipments.csv", index=False)
    new = make_messy(simulate(2, 99), seed=8, n_missing=(3, 2, 1, 2),
                     n_case=6, n_sentinel=0, n_dupes=0, id_start=90001)
    new = new.drop(columns=["delivery_hours"])
    new.to_csv("data/new_shipments.csv", index=False)
    print(len(hubs), len(raw), len(new))
```

File	Rows	What it is
`data/hubs.csv`	24	`hub_id, hub_city, region, hub_tier, hub_daily_capacity` — one row per hub
`data/raw_shipments.csv`	1,224	Messy export of historical parcels, **including** the outcome `delivery_hours`
`data/new_shipments.csv`	48	Messy export of parcels about to be dispatched (two per hub); has **no** `delivery_hours`

Columns of the raw files: `shipment_id`, `hub_id`, `service_type` (Express / Standard / Economy, with inconsistent casing and spacing), `payment_mode` (Prepaid / COD, some blank), `parcel_weight_kg`, `distance_km`, `declared_value` (text such as `₹1,290`, some blank), `n_stops` (stops on the rider's route), `rain_flag` (0/1), `sale_event_flag` (0/1; 1 during the 15–28 September sale window), `dispatch_time` (text, `YYYY-MM-DD HH:MM`), five bookkeeping columns `label_print_batch`, `scanner_build_no`, `tape_roll_no`, `sticker_serial_mod`, `shelf_slot_no`, and (historical file only) `delivery_hours`.

Promise and delay definition. `PROMISED_HOURS = {"Express": 36, "Standard": 60, "Economy": 96}`. A parcel is **delayed** when `delivery_hours > promised_hours`.

Feature list, pinned. Use exactly these 17 columns, in exactly this order, as the model inputs in Parts B, C and D (paste both constants into `src/shipment_prep.py`):

```
PROMISED_HOURS = {"Express": 36, "Standard": 60, "Economy": 96}
FEATURES = [
    "promised_hours", "distance_km", "parcel_weight_kg", "log_declared_value", "n_stops",
    "rain_flag", "sale_event_flag", "is_cod", "is_weekend", "dispatch_hour",
    "hub_tier", "hub_daily_capacity",
    "label_print_batch", "scanner_build_no", "tape_roll_no", "sticker_serial_mod", "shelf_slot_no",
]
NOISE_COLUMNS = ["label_print_batch", "scanner_build_no", "tape_roll_no", "sticker_serial_mod", "shelf_slot_no"]
```

Every CSV you write in `outputs/` and `models/` must be written with `index=False` unless the task says otherwise, and every decimal value rounded to 4 places.

Part A — Data Foundation and Delay Evidence (25 marks)

Part A turns the messy export into a trustworthy modelling table and produces the first evidence about where delays come from. **Parts B, C and D read the table Part A writes, and Part D's scoring script re-uses the two functions you write here.**

Tasks

1. **Repository and data.** Create the repository, commit `generate_data.py`, the three generated files and a `requirements.txt` that pins the four packages to the exact versions you used, one line each with `==` (for example `pandas==2.2.3`; copying the four lines from the output of `pip freeze` is fine): `pandas`, `numpy`, `scikit-learn` and `joblib`.

2. **`clean_shipments(raw, medians=None)`** in `src/shipment_prep.py`. It takes a raw shipments DataFrame and returns a tuple `(clean_df, medians)`. Apply these steps in this order:

    1. drop exact duplicate rows;
    2. `service_type`: strip leading and trailing spaces and convert to title case, so exactly three spellings remain (`Express`, `Standard`, `Economy`);
    3. `payment_mode`: replace a missing value with the placeholder `"Unknown"`;
    4. `declared_value`: remove the rupee sign and the thousands commas, convert to a number, store it in a new column `declared_value_inr`, and drop the text column;
    5. `dispatch_time`: convert to a datetime;
    6. `parcel_weight_kg`: any value above 100 is a scanner-fault sentinel (the network accepts parcels up to 40 kg) — treat it as missing;
    7. fill missing `distance_km`, `parcel_weight_kg` and `declared_value_inr` with that column's **median**. If `medians` is `None`, compute the three medians from the non-missing values left after steps 1–6; otherwise use the dictionary passed in. The returned `medians` is a dictionary with exactly the keys `distance_km`, `parcel_weight_kg`, `declared_value_inr` and float values.

    The medians are computed on the full historical table before any split. They are fill-in values for gaps and carry no information about the target; the scaler and the feature selector in Parts B and C are fitted on training rows only.

3. **`build_features(clean, hubs)`** in `src/shipment_prep.py`. It returns a DataFrame with the hub columns attached by a **left merge** on `hub_id` (`hub_city`, `region`, `hub_tier`, `hub_daily_capacity`) plus these new columns: `promised_hours` (from `PROMISED_HOURS`, integer), `is_cod` (1 when `payment_mode` is `COD`, else 0 — `Unknown` gives 0), `is_weekend` (1 when the dispatch day is Saturday or Sunday), `dispatch_hour` (the hour of the dispatch timestamp as an integer; the generated data uses hours 6 to 21), and `log_declared_value` (natural log of `declared_value_inr`). It must not use `delivery_hours`, so that it also works on parcels that have not been delivered yet.

4. **`part_a_clean_eda.py`.** Read `data/raw_shipments.csv` and `data/hubs.csv`; call your two functions; add the target `is_delayed` (1 when `delivery_hours > promised_hours`, else 0); write the result to `data/modelling_table.csv` (all columns, one row per unique shipment).

5. **Checks file.** The same script writes `outputs/part_a_checks.csv` with two columns, `metric` and `value`, containing the metrics listed in the reference table below, **computed by code from the data — never typed in**. "Before" counts are measured after removing duplicates and before any filling; `missing_parcel_weight_before` does not include the sentinel rows. `service_type_spellings_before` and `service_type_spellings_after` are the number of distinct `service_type` strings in the table (after removing duplicates) before and after the strip-and-title-case step, for example with `nunique()`. `nulls_in_features_and_target` counts missing cells over the 17 `FEATURES` columns plus `delivery_hours` and `is_delayed`. The four declared-value metrics (`declared_value_mean_inr`, `declared_value_median_inr`, `log_declared_value_mean`, `log_declared_value_median`) are computed on the **median-filled** `declared_value_inr` column of the modelling table (and its natural log), not on the non-missing values only. The `hours_*` metrics use the 25th and 75th percentiles of `delivery_hours` and the 1.5 × IQR rule; compute the bounds from the **unrounded** quartiles and round only when writing the file.

6. **Delay evidence table.** Write `outputs/part_a_delay_by_group.csv` with columns `dimension, group, shipments, delayed, delay_rate, mean_hours`, one row per group, for three dimensions: `service_type`; `hub_tier`; and `distance_band`, where `distance_band` is `distance_km` cut into the bins `0-200`, `201-400`, `401-600`, `601+` (edges 0, 200, 400, 600, 2000; right edge inclusive) with those labels. `delay_rate` is the share of delayed parcels in the group; `mean_hours` is the mean of `delivery_hours`.

7. **Correlations.** Write `outputs/part_a_correlations.csv`: the correlation matrix of the numeric columns of the modelling table (row labels kept as the first column).

8. **README — "Data findings".** In `README.md` write at least four findings, **each quoting a number copied from your own output files**: (i) which service type is delayed most often and at what rate; (ii) how delay rate changes from Tier 1 to Tier 3; (iii) how it changes across distance bands; (iv) the three columns most strongly correlated with `delivery_hours` — rank the columns of `part_a_correlations.csv` by absolute correlation, exclude `delivery_hours` itself, and quote each signed value to four decimals; and (v) one sentence on why `declared_value_inr` was log-transformed (compare its mean and median before and after, using the filled column).

Acceptance criteria (Part A is complete when…)

- `generate_data.py` is byte-identical to the script above and the three data files regenerate to the same contents.
- `clean_shipments` and `build_features` have exactly the signatures and return types above; running `clean_shipments` on `data/new_shipments.csv` with the stored medians does not raise and leaves no missing value in the 17 feature columns after `build_features`.
- Every metric in `part_a_checks.csv` matches the reference table; the 1,200 rows survive the hub merge (a left merge must never change the row count here).
- `part_a_delay_by_group.csv` matches the reference group table exactly (counts and delayed totals) and its rates to 4 decimals.
- The `distance_band` bins are exactly the ones specified.
- `modelling_table.csv` has 1,200 rows, contains all 17 `FEATURES` columns plus `delivery_hours` and `is_delayed`, and has no missing value in any of them.
- The README findings each cite a number that appears in your output files.

Reference values for Part A (all *exact*; non-integers to ±0.01)

metric	Value	metric	Value
raw_rows	1224	median_distance_km	314.95
duplicate_rows_removed	24	median_parcel_weight_kg	1.83
rows_after_dedup	1200	median_declared_value_inr	1121.5
missing_distance_km_before	36	rows_after_hub_merge	1200
missing_parcel_weight_before	24	nulls_in_features_and_target	0
sentinel_weight_rows	5	delayed_count	278
missing_declared_value_before	12	delay_rate	0.2317 (±0.0001)
missing_payment_mode_before	18	hours_q1 / hours_q3	43.26 / 72.56
service_type_spellings_before	12	hours_lower_bound / hours_upper_bound	−0.71 / 116.52
service_type_spellings_after	3	hours_outlier_count	1
declared_value_mean_inr	1533.05	declared_value_median_inr	1121.5
log_declared_value_mean	7.028	log_declared_value_median	7.022

dimension	group	shipments	delayed	delay_rate
service_type	Economy	315	27	0.0857
service_type	Express	243	119	0.4897
service_type	Standard	642	132	0.2056
hub_tier	1	400	16	0.0400
hub_tier	2	400	86	0.2150
hub_tier	3	400	176	0.4400
distance_band	0-200	274	6	0.0219

dimension	group	shipments	delayed	delay_rate
distance_band	201-400	563	77	0.1368
distance_band	401-600	275	120	0.4364
distance_band	601+	88	75	0.8523

The four declared-value rows (`declared_value_mean_inr`, `declared_value_median_inr`, `log_declared_value_mean`, `log_declared_value_median`) refer to the median-filled `declared_value_inr` column of the modelling table; the mean of the non-missing values alone differs, so do not compute them before filling.

Correlation spot-checks (±0.01): `corr(delivery_hours, promised_hours)` = 0.837; `corr(delivery_hours, distance_km)` = 0.401; `corr(is_delayed, distance_km)` = 0.546.

Part B — Delivery-Hours Regression (20 marks)

Planners need a point estimate of `delivery_hours` for every parcel. Part B builds it from the table Part A wrote and decides which regression model deserves to be trusted. The leakage-safe habits you practise here (split first, fit the scaler on training rows only) carry straight into Part C.

Tasks — all in `part_b_regression.py`, reading `data/modelling_table.csv`. The inputs `X` are the 17 `FEATURES` columns in the pinned order; the target `y` is `delivery_hours`.

1. **Split into training, validation and test.** Call `train_test_split` with `test_size=0.3, random_state=42` to separate 70% for training from a temporary 30%; call it again on the temporary part with `test_size=0.5, random_state=42` to get validation and test. Sizes must be 840 / 180 / 180.
2. **Scaling without leakage.** Fit a `StandardScaler` on the 840 training rows **only**, then transform the test rows with that fitted scaler. Write `outputs/part_b_checks.csv` (`metric,value`) with `train_rows`, `val_rows`, `test_rows`, `train_scaled_max_abs_mean` (largest absolute column mean of the scaled training rows) and `test_scaled_max_abs_mean` (the same for the scaled test rows).
3. **Baseline.** Predict the mean of `y_train` for every validation row and record its metrics.
4. **Tree-depth diagnostic.** For `DecisionTreeRegressor(max_depth=d, random_state=42)` with `d` in `1, 2, 3, 4, 6, 8, 12` and no limit, fit on the training rows and write `outputs/part_b_tree_depth.csv` with columns `max_depth` (write `unlimited` for no limit), `train_rmse`, `val_rmse`. The depth with the lowest `val_rmse` (ties → smaller depth) is the decision-tree depth used in task 5.
5. **Four candidate models**, each fitted on the training rows and scored on the validation rows with MAE, MSE, RMSE and R²: `LinearRegression` inside a `Pipeline` of `("scaler", StandardScaler())` then `("model", LinearRegression())`; the decision tree from task 4; `RandomForestRegressor(n_estimators=100, random_state=42)`; `GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, random_state=42)`. Write `outputs/part_b_metrics.csv` with columns `model, split, MAE, MSE, RMSE, R2` and rows named `mean_baseline`, `linear_regression`, `decision_tree`, `random_forest`, `gradient_boosting` (all `split = val`), plus the final-test row from task 7. `RMSE` must equal the square root of `MSE` (±0.001).
6. **Learning-rate experiment.** Fit `GradientBoostingRegressor(n_estimators=100, learning_rate=lr, random_state=42)` for `lr` in `0.01, 0.1, 1.0` and write `outputs/part_b_gb_learning_rate.csv` (`learning_rate, val_rmse, val_r2`).
7. **Cross-validated comparison and final model.** Rank the four candidates by validation RMSE. For the **two** best, run 5-fold cross-validation with `cross_val_score` on the 1,020 training-plus-validation rows, using **one** `KFold(n_splits=5, shuffle=True, random_state=42)` object shared by both models (default scoring, which is R²). Write `outputs/part_b_cv.csv` (`model, mean_r2, std_r2`). The **final model** is the candidate with the lowest validation RMSE; score it once on the test rows (already fitted on the training rows) and add the row `model = <its name>, split = test` to `part_b_metrics.csv`.
8. **README — "Model results (regression)".** Report, from your own files: the baseline R² and what it means; which model won on validation and its RMSE in hours; which depths are high-bias and which are high-variance (use the gap between `train_rmse` and `val_rmse`); which learning rate was best and why the other two fail (too slow versus overshooting); and, if the cross-validation ranking disagrees with the validation ranking, which model you would ship and why.

Acceptance criteria

- Split sizes are exactly 840 / 180 / 180; `train_scaled_max_abs_mean` < 1e-9 while `test_scaled_max_abs_mean` is clearly non-zero (> 0.01), proving the scaler was not fitted on the test rows.
- `part_b_metrics.csv` has the six rows described; every `RMSE` is √`MSE`; no model is scored on the test rows except the final one, and only once.
- `part_b_tree_depth.csv` has 8 rows; `train_rmse` never rises as depth grows; the figures satisfy the bands below; the `decision_tree` row of `part_b_metrics.csv` has a validation RMSE equal (within 0.001) to the lowest `val_rmse` in `part_b_tree_depth.csv`.
- `part_b_gb_learning_rate.csv` has 3 rows and satisfies R²(0.1) > R²(1.0) > R²(0.01).
- Both cross-validated models were evaluated with the same `KFold` object, and `part_b_cv.csv` has 2 rows.
- The README statements agree with the numbers in the CSV files.

Reference values for Part B (pinned configuration → pass band)

Item	Reference	Pass band
Baseline R^2 (validation)	−0.021	−0.10 to 0.02
Linear regression R^2 (validation)	0.931	≥ 0.88
Gradient boosting R^2 (validation)	0.931	≥ 0.88
Random forest R^2 (validation)	0.914	≥ 0.86
Decision tree R^2 (validation)	0.887	≥ 0.82
Best tree depth / its `val_rmse`	6 / 6.28	best depth is 4 or more, or unlimited
Depth 1: `val_rmse` − `train_rmse`	0.50	≤ 2.5 (high bias: both errors high)
No depth limit: `train_rmse` / `val_rmse` − `train_rmse`	0.00 / 7.79	gap ≥ 5.0 (high variance)
Depth 1 `val_rmse` minus best `val_rmse`	5.85	≥ 4.0
Learning rate 0.01 / 0.1 / 1.0: validation R^2	0.713 / 0.931 / 0.844	$R^2(0.1) \geq 0.90$; $R^2(0.01) \leq 0.80$
Cross-validated mean R^2 (each of the two models)	0.926 and 0.935	≥ 0.90; `std_r2` ≤ 0.03
Final model R^2 on test	0.949	≥ 0.88

The identity of the winning model (linear regression versus gradient boosting) can legitimately change with the split, so it is **not** a graded value — the justification is.

Part C — Delay Classifier and Alert Threshold (30 marks)

Now the sharper question: will this parcel miss its promise? About 23% of parcels do, so accuracy alone can hide a useless model. Part C builds the classifier, removes the bookkeeping columns that carry no signal, tunes the forest, and — most importantly — picks the probability threshold that matches the operations rule.

Tasks — all in `part_c_classification.py`, reading `data/modelling_table.csv`. Inputs are the 17 `FEATURES` in the pinned order; the target is `is_delayed`.

1. **Stratified split.** Repeat Part B's two-step split (`test_size=0.3` then `test_size=0.5`, `random_state=42` both times) with `stratify` on the target in **both** calls. Expected sizes 840 / 180 / 180 with 195 / 42 / 41 delayed parcels.
2. **Blind baseline and three classifiers**, scored on the validation rows with the default 0.5 decision rule. Write `outputs/part_c_models.csv` with columns `model, TN, FP, FN, TP, accuracy, precision, recall, f1` and exactly these rows: `blind_all_on_time` (predicts "not delayed" for every parcel; write precision as 0); `logistic_regression` (a `Pipeline` of `StandardScaler` then `LogisticRegression`); `decision_tree_depth_<d>` (`DecisionTreeClassifier(criterion="gini", max_depth=d, random_state=42)`, with `d` chosen from `2, 3, 4, 5, 6, 8, 10` by highest validation F1, ties → smaller `d`, and `<d>` replaced by the chosen value); `random_forest_default` (`RandomForestClassifier(n_estimators=100, random_state=42)`). The four confusion-matrix cells in each row must reproduce `accuracy`, `precision`, `recall` and `f1`.
3. **Which columns carry signal?** Fit a default forest (`n_estimators=100, random_state=42`) on all 17 columns of the training rows to get `feature_importances_`; fit `SelectKBest(f_classif, k="all")` on the training rows to get one F-score per column. Write `outputs/part_c_feature_scores.csv` (`feature, f_score, f_rank, rf_importance, is_noise`; `f_rank` 1 = highest F-score; `is_noise` is `True` for the five `NOISE_COLUMNS`). Then, for `k` in `5, 7, 10, 12, 17`, fit a `Pipeline` of `SelectKBest(f_classif, k=k)` followed by `RandomForestClassifier(n_estimators=100, random_state=42)` on the training rows and write the validation F1 to `outputs/part_c_selectk.csv` (`k, val_f1`). **From here on `k = 7`.**
4. **Tuning.** Fit `SelectKBest(f_classif, k=7)` on the training rows only and transform the training and validation rows with it. On the transformed training rows run, for `RandomForestClassifier(random_state=42)`, over the grid `n_estimators` ∈ {50, 100, 150}, `max_depth` ∈ {4, 6, 8}, `min_samples_leaf` ∈ {1, 5, 10}: `GridSearchCV(..., cv=5)` and `RandomizedSearchCV(..., n_iter=9, cv=5, random_state=42)` (default scoring for both). Write `outputs/part_c_tuning.csv` with columns `search, candidates, cv_folds, total_fits, best_params, best_cv_accuracy, val_f1` and three rows: `grid_search`, `randomized_search`, `default_forest_same_7_features` (the untuned forest `n_estimators=100`, no depth limit, `min_samples_leaf=1` on the same 7 columns; `candidates` 1, `cv_folds` 0, `total_fits` 1, `best_params` = `defaults`, `best_cv_accuracy` left blank). `total_fits` = `candidates × cv_folds` for the two searches; `val_f1` is each candidate's F1 on the validation rows. The test rows must not be touched in this task.
5. **Choosing the final configuration.** Pick the row of `part_c_tuning.csv` with the highest `val_f1` (ties → `default_forest_same_7_features`, then `grid_search`, then `randomized_search`). The final model is a `Pipeline([("selector", SelectKBest(f_classif, k=7)), ("model", RandomForestClassifier(random_state=42, <chosen parameters>))])` fitted on the training rows.

6. **Alert threshold.** Take the final model's validation probabilities (`predict_proba`, positive class). Build the thresholds as exact decimals — use the literal list `[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]` or round each value to one decimal (a floating-point range such as `np.arange(0.1, 1.0, 0.1)` produces values like `0.30000000000000004` that change the result at the boundary). For each threshold `t`, predict "delayed" when the probability is ≥ `t`, and write `outputs/part_c_thresholds.csv` with columns `threshold, TN, FP, FN, TP, accuracy, precision, recall, f1`. **Operations rule:** because a missed delayed parcel costs more than an unnecessary call, the alert threshold is the **highest `t` whose validation recall is at least 0.80** (if none reaches 0.80, use 0.1).
7. **One test evaluation.** Score the final model on the **test** rows at the chosen threshold, exactly once, and write `outputs/part_c_final_test.csv` (`threshold, TN, FP, FN, TP, accuracy, precision, recall, f1`). Write `outputs/part_c_final_config.csv`, one row, columns `selector_k, n_estimators, max_depth, min_samples_leaf, threshold, chosen_from` (`max_depth` is `unlimited` when there is no limit; `chosen_from` is the name of the winning candidate in `part_c_tuning.csv`).
8. **README — "Model results (classification)".** From your own files: the blind baseline's accuracy and why it is useless; the noise columns' combined share of the forest's importance and whether all five fall outside the top 7 by F-score; what the `k` table shows; whether tuning beat the untuned forest on validation, and what that says about tuning on a 180-row validation set; the chosen threshold and the validation precision/recall trade-off behind it; and the test precision, recall and F1, comparing test recall with validation recall honestly (the test set has only 41 delayed parcels).

Acceptance criteria

- Split sizes and delayed counts are exactly as stated in task 1.
- Every row in `part_c_models.csv` is internally consistent (accuracy recomputed from the four cells equals `accuracy`), and the blind row is `TN 138, FP 0, FN 42, TP 0`, accuracy 0.7667.
- `part_c_feature_scores.csv` has 17 rows. The seven columns ranked 1–7 are exactly `distance_km`, `hub_tier`, `hub_daily_capacity`, `n_stops`, `promised_hours`, `rain_flag`, `sale_event_flag`; every `NOISE_COLUMNS` entry has `f_rank` ≥ 8 and an F-score below 10; on the pinned split the smallest F-score among the top seven is above 30 (the noise columns stay below 10).
- `part_c_selectk.csv` has 5 rows.
- `part_c_tuning.csv`: `candidates` = 27 and 9, `total_fits` = 135 and 45; `best_cv_accuracy` of the grid ≥ that of the randomized search; the selector in task 4 was fitted on training rows only.
- `part_c_thresholds.csv` has 9 rows, `recall` never increases as `threshold` rises, and the chosen threshold in `part_c_final_config.csv` is the one the operations rule produces **from your own table**.
- `part_c_final_test.csv` has one row and its threshold equals the chosen one; the model was not re-tuned after seeing it.

Reference values for Part C (pinned configuration → pass band)

Item	Reference	Pass band
Delayed parcels in train / validation / test	195 / 42 / 41	exact
Blind baseline accuracy (validation)	0.7667	exact (±0.001)
Logistic regression: F1 (validation)	0.805	≥ 0.65
Best decision tree: depth / F1	6 / 0.747	F1 ≥ 0.60
Default random forest F1 (validation)	0.800	≥ 0.65
Accuracy of every trained model (validation)	0.883–0.911	above the blind baseline's 0.7667
Noise columns' share of forest importance (17 columns)	0.225	0.15 to 0.30
Grid / randomized best `best_cv_accuracy`	0.8952 / 0.8905	each 0.86 to 0.93
`val_f1` of each row in `part_c_tuning.csv`	0.775 / 0.790 / 0.833	each ≥ 0.65
Chosen threshold; test precision / recall / F1	0.5; 0.689 / 0.756 / 0.721	threshold per rule; F1 ≥ 0.60, recall ≥ 0.55, precision ≥ 0.45

The configuration chosen in task 5 and the resulting threshold may legitimately differ between valid implementations (the three validation F1 scores are close); they are graded by consistency with your own tables, not by value.

Part D — Hub Segments, Packaging and Operations Playbook (25 marks)

Part D finishes the build. It clusters the 24 hubs into operating segments, finds hubs whose tier label is misleading, saves the final classifier as a reusable artefact, and ships a scoring script that turns the 48 new parcels into a concrete action list by combining Part C's probabilities with Part D's segments.

Tasks

1. **Hub profile** (`part_d_segments.py`). From `data/modelling_table.csv`, group by `hub_id` and compute `shipments` (count), `mean_delivery_hours`, `delay_rate` (share of delayed parcels), `mean_distance_km` and `mean_n_stops`. Write `outputs/hub_profile.csv` (24 rows, sorted by ascending `hub_id`; columns `hub_id` plus those five).
2. **K-Means segmentation.** Scale the four profile columns `mean_delivery_hours, delay_rate, mean_distance_km, mean_n_stops` with `StandardScaler` (distance-based clustering needs comparable scales) and fit `KMeans(n_clusters=3, random_state=42, n_init=10)`. **Fixed order of the hubs:** the hub profile is sorted by **ascending `hub_id`** (HUB01 first, HUB24 last — the order `groupby` returns and the order of `hub_profile.csv`) before it is scaled and clustered, so the 24 hubs always enter `StandardScaler` and `KMeans` in that order; the four profile columns may be in any order. **K = 3 is fixed by the business** — operations runs exactly three playbooks.
3. **Separation evidence.** For `K` in 2, 3, 4, 5 (each `KMeans(n_clusters=K, random_state=42, n_init=10)` on the same scaled matrix, hubs in ascending `hub_id` order) compute the **across-to-within distance ratio**: the mean Euclidean distance over all unordered hub pairs that sit in *different* clusters divided by the mean over pairs in the *same* cluster, measured on the scaled matrix (276 pairs). Write `outputs/part_d_separation.csv` with columns `K, across_over_within, cluster_sizes`, where `cluster_sizes` is the cluster sizes in descending order joined by hyphens (for example `9-8-7`).
4. **Segment names and tier check.** Order the three clusters by the mean of their hubs' `delay_rate` and name them `Standard` (lowest), `Watch` (middle) and `Escalate` (highest). Write `outputs/hub_segments.csv` (`hub_id, segment, hub_city, hub_tier`), `outputs/segment_profile.csv` (`segment, hubs, mean_delivery_hours, delay_rate, mean_distance_km, mean_n_stops` — means of the hub-level values) and `outputs/tier_vs_segment_mismatch.csv` (the rows of `hub_segments.csv` whose segment differs from the one their tier implies: Tier 1 → `Standard`, Tier 2 → `Watch`, Tier 3 → `Escalate`).
5. **Package the model** (`package_model.py`). Read `outputs/part_c_final_config.csv`, rebuild Part C's stratified training/validation/test split with the same settings, fit the final `Pipeline` on the training rows, and save with `joblib`: `models/delay_model.pkl`; `models/delay_threshold.csv` (one column `threshold`); `models/feature_order.csv` (one column `feature`, the 17 names in `FEATURES` order); and `models/impute_values.csv` (columns `column, median` — the three medians from `clean_shipments`). Then reload the pickle with `joblib.load`, predict probabilities for the 180 test rows with both the reloaded and the in-memory pipeline, and write `outputs/part_d_reload_check.csv` (`metric,value`) with `test_rows_scored`, `reload_max_abs_probability_diff` and `threshold`.
6. **Scoring script** (`predict_delay.py`). Load `data/new_shipments.csv`, `data/hubs.csv`, the four files in `models/` and `outputs/hub_segments.csv`; clean the new parcels with **your own `clean_shipments`, passing the stored medians**; attach features with **your own `build_features`**; arrange the columns in the order of `feature_order.csv`; compute `delay_probability` (positive class, rounded to 4 decimals); set `flagged = 1` when the probability is ≥ the stored threshold; join the hub segment; and set `action` to `Escalate to hub manager` when the parcel is flagged **and** its hub's segment is `Escalate`, `Proactive customer call` when it is flagged otherwise, and `No action` when it is not flagged. Write `outputs/new_shipment_actions.csv` with columns `shipment_id, hub_id, segment, delay_probability, flagged, action`, one row per new parcel, in file order. Running the script twice must produce byte-identical output.
7. **README — "How to run", "Operations recommendation", "Production plan" and "Limitations".**
    - *How to run:* the exact commands, in order, from a fresh clone, including `pip install -r requirements.txt`. State that `models/delay_model.pkl` is regenerated by running `python package_model.py`; the committed pickle loads reliably only on the scikit-learn version pinned in `requirements.txt`, and the grader re-runs `package_model.py` instead of loading the committed pickle when the installed versions differ.
    - *Operations recommendation* for the regional head: the hubs in the `Escalate` segment with their delay rate; every hub in `tier_vs_segment_mismatch.csv` and what that means for how its tier label should be used; the alert threshold, and — from `part_c_final_test.csv` — how many of every 10 flagged parcels are really delayed (precision) and how many of every 10 delayed parcels are caught (recall); and how many of the 48 new parcels were flagged and how many of those need a hub-manager escalation; and one or two sentences explaining why you kept K = 3 for the segments, citing your `part_d_separation.csv` (see the K-Means stability note in task 3).
    - *Production plan:* (a) map **at least five** of these seven production-readiness characteristics to a concrete item in your repository — accuracy, reliability, reproducibility, performance (speed), maintainability, consistency (same preprocessing in training and prediction), monitorability; (b) name **at least three** quantities to monitor with a numeric trigger for each (for example the weekly delayed share of delivered parcels, the weekly share of parcels flagged, and recall measured on parcels whose outcome is now known); (c) give a retraining schedule and say how the 15–28 September sale window (`sale_event_flag`) should be handled — compare against the same kind of event, not against ordinary weeks — and what data drift would look like here.
    - *Limitations:* at least three honest limitations (for example synthetic data, the small number of delayed parcels in the test set, and a threshold tuned on one validation split).

Acceptance criteria

- `hub_profile.csv` has 24 rows and every `shipments` value is 50; the spot checks in the reference table match.
- `part_d_separation.csv` has 4 rows. K = 2 and K = 3 match the reference sizes (exact) and ratios (±0.01). K = 4 and K = 5 match the reference sizes and ratios (±0.01) when the pinned hub order and `random_state=42` are used; if a different hub order or seed was used (which also forfeits the K-Means setup marks) they are graded on consistency with the student's own file and not on the reference sizes or ratios. The README's Operations

recommendation section explains why K = 3 is kept although the ratio keeps rising for K = 4 and K = 5 (those solutions contain a cluster of fewer than 6 hubs, too small to carry its own playbook).

- Segment sizes and profiles match the reference; `tier_vs_segment_mismatch.csv` lists exactly the four reference hubs; the segment *names* follow the delay-rate ordering rule, not the arbitrary cluster numbers.
- `part_d_reload_check.csv`: `test_rows_scored` = 180 and `reload_max_abs_probability_diff` ≤ 1e-9; the four `models/` files exist and `feature_order.csv` equals the pinned feature list in order.
- `new_shipment_actions.csv` has 48 rows with unique `shipment_id`s and no missing value; `flagged` is 1 exactly when `delay_probability` ≥ the stored threshold; every `action` follows the rule; `predict_delay.py` uses the shared functions from `src/shipment_prep.py` rather than a second copy of the cleaning logic; two consecutive runs produce identical files.
- `requirements.txt` pins `pandas`, `numpy`, `scikit-learn` and `joblib` with `==`, and the README's *How to run* section says that `package_model.py` regenerates `models/delay_model.pkl`.
- The README contains all six sections named in this brief (`How to run`, `Data findings`, `Model results` for regression and classification, `Operations recommendation`, `Production plan`, `Limitations`), and every number it quotes appears in the repository's output files.

Reference values for Part D

Item	Reference	Tolerance
`hub_profile.csv`, `HUB01`	hours 45.685 · delay rate 0.0200 · distance 185.061 · stops 54.70	±0.01
`hub_profile.csv`, `HUB13`	hours 66.074 · delay rate 0.4600 · distance 473.320 · stops 83.92	±0.01
`hub_profile.csv`, `HUB17`	hours 60.851 · delay rate 0.4600 · distance 443.285 · stops 84.18	±0.01
`part_d_separation.csv` K = 2 / 3: `cluster_sizes`	`15-9` / `9-8-7`	exact
`part_d_separation.csv` K = 2 / 3: `across_over_within`	2.894 / 5.393	±0.01
`part_d_separation.csv` K = 4 / 5: `cluster_sizes`	`8-7-5-4` / `8-7-4-3-2`	exact for `random_state=42`, `n_init=10` and ascending `hub_id` order; see the note below the table
`part_d_separation.csv` K = 4 / 5: `across_over_within`	6.305 / 6.887	±0.01 for the pinned order and seed; otherwise graded on consistency with your own file
Segment `Standard`: hubs · hours · delay rate · distance · stops	7 · 47.24 · 0.0257 · 188.93 · 55.37	±0.01
Segment `Watch`	8 · 56.81 · 0.1550 · 314.18 · 70.72	±0.01
Segment `Escalate`	9 · 64.65 · 0.4600 · 470.48 · 84.34	±0.01
Tier-versus-segment mismatches	`HUB07` Kolkata (Tier 1, `Watch`); `HUB13` Nagpur (Tier 2, `Escalate`); `HUB15` Kochi (Tier 2, `Escalate`); `HUB22` Dehradun (Tier 3, `Watch`)	exact
`new_shipments.csv` before cleaning	48 rows · 3 missing `distance_km`, 2 missing `parcel_weight_kg`, 1 missing `declared_value`, 2 missing `payment_mode`, 8 `service_type` spellings	exact
`reload_max_abs_probability_diff`	0.0	≤ 1e-9

Stability of the K-Means results. K = 2 and K = 3 were identical for every random seed (200 tried) and every hub order (1,000 shuffled orders tried) and for all 24 orderings of the four profile columns. K = 4 and K = 5 are exact for `random_state=42`, `n_init=10` and the ascending-`hub_id` order stated above, for all 24 column orderings, and for a profile read back from the 4-decimal `hub_profile.csv`. A different seed or hub order can land on a nearby local optimum (K = 4 with ratio 6.21, or rarely 6.06 with sizes `8-7-6-3`; K = 5 with sizes such as `8-7-5-2-2` at ratio 6.755, `8-7-4-4-1` at 6.788, `8-7-4-3-2` at 6.777, or rarely `7-6-5-4-2` at 6.51). Because the order and seed are mandatory above, the reference sizes and ratios apply to you. If you deviate (which also forfeits the K-Means setup marks), your K = 4 and K = 5 rows are graded on agreement between `cluster_sizes` and your own `part_d_separation.csv` and on the README argument that each of K = 4 and K = 5 contains a cluster of fewer than 6 hubs; that argument held in about 10,000 runs with different orders and seeds.

Submission

Submit **one public GitHub repository link**. The repository must contain, in total: `generate_data.py` and the generated `data/` files; `src/shipment_prep.py`; the scripts `part_a_clean_eda.py`, `part_b_regression.py`, `part_c_classification.py`, `part_d_segments.py`, `package_model.py` and `predict_delay.py`; the `models/` and `outputs/` files named in this brief; `requirements.txt` (the four packages pinned with

==); and **one `README.md`** with the sections *How to run*, *Data findings*, *Model results* (regression and classification), *Operations recommendation*, *Production plan* and *Limitations*.

All work must live on one branch — `main` — and the submitted link must point to `main`. The accepted forms are the repository root URL `https://github.com/<user>/<repo>` (only when `main` is the repository's default branch) or `https://github.com/<user>/<repo>/tree/main`; check that the default branch is `main` before you submit. A link to any other branch, or a link that does not clearly resolve to `main`, **will not be graded**. No file other than the repository itself is submitted: no images, screenshots, PDFs, slides, presentations, videos or audio.